ASSIGNMENT 4 – CO4

Designing Reusable Class Hierarchies

Object Oriented Programming with C++ (DSA01)

Name: TELAPROLU ROHITA SAI

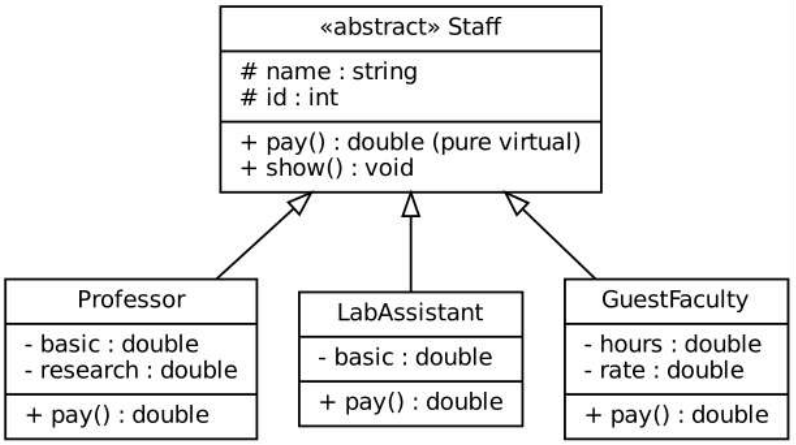
Register No: 192425041

TASK 1 – Class Design Review

1.1 Diagnosis (deficiencies)

- Repeated fields: name, id and basic appear in all three classes; the same data is declared three times.
- Repeated pay rule: Professor and Lab Assistant both compute basic + basic * 0.42 separately.
- Public data members: any code can change salary data directly (no encapsulation).
- Inconsistent names: Guest Faculty uses nm/code while the others use name/id for the same information.
- The da field is declared but never used; pay() hard-codes 0.42 instead.
- Magic number 0.42 is written twice; changing the DA rate means editing every class.
- Adding a new staff category means writing another full class and copying everything again.
- No common type, so all staff cannot be stored in one array or processed in one loop.

1.2 Repair – UML and code




```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
const double DA = 0.42;
class Staff {
protected:
    string name; int id;
public:
    Staff(string n, int i) : name(n), id(i) {}
    virtual double pay() const = 0;
    virtual ~Staff() = default;
    void show() const { cout << id << ' ' << name << ' ' << pay() << '\n'; }
};
class Professor : public Staff {
    double basic, research;
public:
    Professor(string n, int i, double b, double r) : Staff(n, i), basic(b), research(r) {}
    double pay() const override { return basic * (1 + DA) + research; }
};
class LabAssistant : public Staff {
    double basic;
public:
```

```
    LabAssistant(string n, int i, double b) : Staff(n, i), basic(b) {}
    double pay() const override { return basic * (1 + DA); }
};
class GuestFaculty : public Staff {
    double hours, rate;
public:
    GuestFaculty(string n, int i, double h, double r) : Staff(n, i), hours(h), rate(r) {}
    double pay() const override { return hours * rate; }
};
int main() {
    vector<Staff*> v = { new Professor("Asha", 1, 50000, 5000), new LabAssistant("Ravi", 2, 20000),
                         new GuestFaculty("Meena", 3, 10, 800) };
    for (auto s : v) { s->show(); delete s; }
}
```

Output:

```
1 Asha 76000
2 Ravi 28400
3 Meena 8000
```

1.3 Design defence (under 100 words)

Common data (name, id) and show() are written once in Staff. Each category only adds its own fields and overrides pay(). A new category, such as VisitingFaculty, is a new derived class only; Staff and the other classes are not edited. Because every object is a Staff, all of them can sit in one vector<Staff*> and be paid in one loop through virtual dispatch, so the payroll loop never changes either.

Alternative Concept and Complexity Analysis

Single Staff class with a type field and a switch inside pay():

```
struct Staff { int type; string name; int id; double basic, research, hours, rate;
  double pay() const { switch (type) {
    case 0: return basic*1.42 + research;   // professor
    case 1: return basic*1.42;              // lab assistant
    default: return hours*rate; } } };      // guest
```

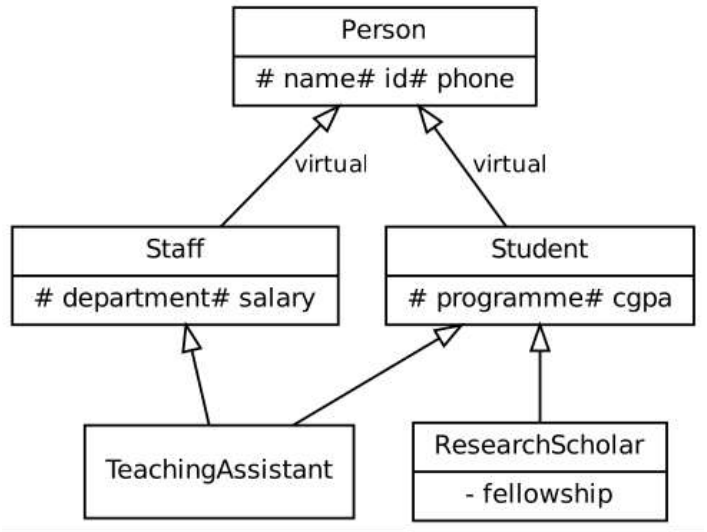
Measure	Base + derived classes	Single class + switch
Time per pay()	O(1) (one virtual call)	O(1) (one switch jump)
Space per object (measured sizeof)	Professor 64, LabAssistant 56, GuestFaculty 64 bytes	72 bytes for every object (unused fields)
Places changed for a new type	1 new class (0 existing edited)	3: type list, new fields, switch case (existing class edited)
Overall space for n objects	O(n)	O(n), with wasted bytes per object

TASK 2 – From Requirement to Hierarchy

2.1 Relationships

Relationship	Type	Inheritance kind
Student is-a Person	is-a	Single
Staff is-a Person	is-a	Single (Person has two children: hierarchical)
TeachingAssistant is-a Student and is-a Staff	is-a	Multiple
TA → Student/Staff → Person (whole structure)	is-a	Hybrid (hierarchical + multiple = diamond)
ResearchScholar is-a Student	is-a	Single; Person→Student→ResearchScholar is multilevel

ResearchScholar has a fellowship	has-a	Member (data member, not inheritance)



2.2 Resolving the diamond

TeachingAssistant inherits Student and Staff, and each of them inherits Person. Without virtual, a TA object holds two separate Person sub-objects. Person's constructor runs twice, memory is wasted, and a name written as t.name is ambiguous (the compiler cannot decide between Student::Person::name and Staff::Person::name). Declaring Student and Staff as virtual Person makes them share one Person sub-object, built once by the most-derived class.

2.3 Class declarations and inheritance mode

Student and Staff inherit Person publicly and virtually (a student is-a person; public keeps Person's interface visible). TeachingAssistant inherits both publicly. ResearchScholar inherits Student publicly. Protected data allows derived classes access while hiding it from outside code.

```
#include <iostream>
#include <string>
using namespace std;
class Person {
protected:
    string name; int id; string phone;
public:
    Person(string n = "", int i = 0, string p = "") : name(n), id(i), phone(p) {}
};
class Student : virtual public Person {
protected:
    string prog; double cgpa;
public:
    Student(string n = "", int i = 0, string p = "", string pr = "", double c = 0) : Person(n, i,
p), prog(pr), cgpa(c) {}
};
class Staff : virtual public Person {
protected:
    string dept; double salary;
public:
    Staff(string n = "", int i = 0, string p = "", string d = "", double s = 0) : Person(n, i, p),
dept(d), salary(s) {}
};
class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string p, string pr, double c, string d, double s)
        : Person(n, i, p), Student(n, i, p, pr, c), Staff(n, i, p, d, s) {}
    void show() const { cout << name << ' ' << id << ' ' << prog << ' ' << dept << '\n'; }
};
class ResearchScholar : public Student {
    double fellowship;
public:
```

```
    ResearchScholar(string n, int i, string p, string pr, double c, double f) : Person(n, i, p),
Student(n, i, p, pr, c), fellowship(f) {}
};
int main() {
    TeachingAssistant t("Kavya", 7, "98765", "MTech", 8.9, "CSE", 25000);
    t.show();
    cout << sizeof(Person) << ' ' << sizeof(Student) << ' ' << sizeof(TeachingAssistant) << '\n';
}
```

Output:

```
Kavya 7 MTech CSE
72 120 168
```

2.4 Access table (member of Person seen inside Student)

Member in Person	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

Predict the Output

My predicted output: Person Student Staff TA

Actual output after running:

Output:

```
Person Student Staff TA
```

Both match. Person is printed only once because Student and Staff share one virtual Person sub-object, which the most-derived class (TeachingAssistant) constructs first, before Student and Staff. Their own Person() calls are skipped.

Without virtual: predicted and actual output is Person Student Person Staff TA (Person is built twice, once per base). The program still compiles, since no ambiguous member is used. If a member is accessed (t.name), g++ gives:

```
error: request for member 'name' is ambiguous
note: candidates are: 'int Person::name'
note:                 'int Person::name'
```

Alternative Concept and Complexity Analysis

TeachingAssistant as composition: struct TA { Student s; Staff f; }; (has-a). This also keeps two Person copies, but each is reached through its own member.

```
#include <iostream>
#include <string>
using namespace std;
struct Person { string name; int id; string phone; };
struct Student : Person { string prog; double cgpa; };
struct Staff : Person { string dept; double salary; };
struct TAcomp { Student s; Staff f; };           // has-a
struct TAnv : Student, Staff {};                  // non-virtual
struct SV : virtual Person { string prog; double cgpa; };
struct FV : virtual Person { string dept; double salary; };
struct TAv : SV, FV {};
int main() {
    cout << "composition " << sizeof(TAcomp) << "  non-virtual " << sizeof(TAnv) << "  virtual " <<
sizeof(TAv) << '\n';
    TAcomp c; c.s.name = "A"; c.f.name = "B"; cout << c.s.name << c.f.name << '\n';   // two names
    TAv v; v.name = "X"; cout << v.name << '\n';                                      // one name
}
```

Output:

```
composition 224  non-virtual 224  virtual 168
AB
X
```

Measure	Virtual diamond	Composition (has-a)
sizeof (measured)	168 bytes (one Person)	224 bytes (two Person)
Reaching name	t.name, one copy, O(1), via virtual-base offset (one extra indirection)	t.s.name or t.f.name, O(1), direct offset; two copies may differ
Ambiguity	None	None, but a TA is not a Student or Staff, so it cannot be passed as one

TASK 3 – Constructors in Derived Classes and Member Classes

3.1 Passing values to base constructors

Each constructor passes values to its base using an initializer list (Student(...) : Person(n, c)). The virtual base has no single parent that owns it, so the most-derived class (TA) must call Person(n, c) itself. Student's and Staff's calls to Person are ignored when they are built as part of a TA.

3.2 – 3.3 Member classes and construction / destruction trace

Address is a member of Person and Course is a member of Student; their constructors receive city and title through the initializer lists.

```
#include <iostream>
#include <string>
using namespace std;
struct Address { string city; Address(string c) : city(c) { cout << "Address "; } ~Address() { cout
<< "~Address "; } };
struct Course { string title; Course(string t) : title(t) { cout << "Course "; } ~Course() { cout <<
"~Course "; } };
struct Person {
    string name; Address addr;
    Person(string n, string c) : name(n), addr(c) { cout << "Person "; }
    ~Person() { cout << "~Person "; }
};
struct Student : virtual Person {
    Course crs;
    Student(string n, string c, string t) : Person(n, c), crs(t) { cout << "Student "; }
    ~Student() { cout << "~Student "; }
};
struct Staff : virtual Person {
    Staff(string n, string c) : Person(n, c) { cout << "Staff "; }
    ~Staff() { cout << "~Staff "; }
};
struct TA : Student, Staff {
    TA(string n, string c, string t) : Person(n, c), Student(n, c, t), Staff(n, c) { cout << "TA
"; }
    ~TA() { cout << "~TA "; }
};
int main() {
    { cout << "Student: "; Student s("A", "Salem", "C++"); cout << "| "; } cout << '\n';
    { cout << "Staff:   "; Staff f("B", "Erode"); cout << "| "; } cout << '\n';
    { cout << "TA:      "; TA t("C", "Chennai", "DSA"); cout << "| "; } cout << '\n';
    cout << "sizeof(TA)=" << sizeof(TA) << '\n';
}
```

Output:

```
Student: Address Person Course Student | ~Student ~Course ~Person ~Address
Staff:   Address Person Staff | ~Staff ~Person ~Address
TA:      Address Person Course Student Staff TA | ~TA ~Staff ~Student ~Course ~Person ~Address
sizeof(TA)=112
```

Rule observed: construction goes virtual base → non-virtual bases (left to right) → member objects (declaration order) → the class's own body. Destruction runs in exact reverse order.

3.4 Experiment (virtual removed)

With the virtual keyword removed from Student and Staff, a TA contains two Person (and two Address) sub-objects. The trace becomes:

Output:

```
TA:      Address Person Course Student Address Person Staff TA | ~TA ~Staff ~Person ~Address
~Student ~Course ~Person ~Address
sizeof(TA)=160
```

sizeof(TeachingAssistant) grows from 112 (virtual) to 160 bytes (non-virtual) because the second Person and Address are stored.

Predict the Output

My predicted output: Vehicle Engine Car | ~Car ~Engine ~Vehicle

```
#include <iostream>
struct Engine { Engine() { std::cout << "Engine "; } ~Engine() { std::cout << "~Engine "; } };
struct Vehicle { Vehicle() { std::cout << "Vehicle "; } ~Vehicle() { std::cout << "~Vehicle "; } };
struct Car : Vehicle {
    Engine e;
    Car() { std::cout << "Car "; }
    ~Car() { std::cout << "~Car "; }
};
int main() { { Car c; std::cout << "| "; } std::cout << '\n'; }
```

Output:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Matches. The base class Vehicle is built first, then the member Engine, then the Car body. Destruction runs in the opposite order: ~Car, ~Engine, ~Vehicle.

Complete the Code (TODO 1)

Constructor added inside class Student:

```
Student(const std::string& n, int a, long long r, const std::string& c, int p)
    : Person(n, a), regNo(r), home(c, p) {}
```

Output:

```
192311001 Kavya (19) Salem-636001
```

Swapping the order of the initializer list (home first, Person last) and compiling with -Wall gives:

```
warning: 'Student::home' will be initialized after [-Wreorder]
warning:   'long long int Student::regNo' [-Wreorder]
```

Members are always initialised in declaration order (base first, then regNo, then home), never in the written order of the list.

Alternative Concept and Complexity Analysis

Student holds Address* home = new Address(...). Constructor allocates with new; the destructor must delete it; the copy constructor must make a deep copy (new Address(*o.home)); the copy assignment must do the same (Rule of Three).

```
#include <iostream>
#include <string>
class Address { std::string city; int pin;
public: Address(const std::string& c, int p) : city(c), pin(p) {} std::string str() const { return
city + "-" + std::to_string(pin); } };
class Student {
    long long regNo; Address* home;
```

```
public:
    Student(long long r, const std::string& c, int p) : regNo(r), home(new Address(c, p)) {}
    Student(const Student& o) : regNo(o.regNo), home(new Address(*o.home)) {}     // deep copy
    Student& operator=(const Student& o) { if (this != &o) { delete home; home = new
Address(*o.home); regNo = o.regNo; } return *this; }
    ~Student() { delete home; }
    void print() const { std::cout << regNo << ' ' << home->str() << '\n'; }
};
int main() { Student s(192311001, "Salem", 636001); Student t = s; t.print(); std::cout <<
sizeof(Address) << ' ' << sizeof(Address*) << '\n'; }
```

Output:

```
192311001 Salem-636001
40 8
```

Measure	Member object	Pointer to Address
Extra time	none (built inline)	one heap allocation, O(1) but slower; one extra indirection per access
Space	40 bytes inside the Student	8 bytes for the pointer, plus 40 bytes on the heap
Special functions	compiler-generated ones are correct	destructor, copy constructor, copy assignment must be written

TASK 4 – Pointers to Derived Classes: Forensics

4.1 Defects

- Senior rate never used: rate() is not virtual, so a->rate() through an Account* is bound at compile time to Account::rate() and returns 3.5.
- Memory leak: ~Account() is not virtual, so delete a on an Account* that points to a SeniorAccount runs only ~Account(). ~SeniorAccount() never runs, and its new double[100] (800 bytes) is never released. This is also undefined behaviour.
- Slicing: Account copy = SeniorAccount(); copies only the Account part. The history pointer and the Senior behaviour are cut off, and copy.rate() always returns 3.5 (same for vector<Account>).

4.2 Evidence

```
#include <iostream>
#include <vector>
class Account { public: double rate() const { return 3.5; } ~Account() { std::cout <<
"~Account\n"; } };
class SeniorAccount : public Account {
    double* history = new double[100];
public: double rate() const { return 4.0; } ~SeniorAccount() { delete[] history; std::cout <<
"~Senior\n"; } };
int main() {
    Account* accounts[2] = { new Account, new SeniorAccount };
    for (Account* a : accounts) std::cout << a->rate() << "\n";
    for (Account* a : accounts) delete a;
    Account copy = SeniorAccount();
}
```

Plain run (destructor output): ~Senior appears only for the temporary in the last line, never for the heap object.

Output:

```
3.5
3.5
~Account
~Account
~Senior
~Account
~Account
```

Compiled with -fsanitize=address, AddressSanitizer stops at the first delete:

```
ERROR: AddressSanitizer: new-delete-type-mismatch
  object passed to delete has wrong type:
  size of the allocated type:   8 bytes;
  size of the deallocated type: 1 bytes.
```

With ASAN_OPTIONS=new_delete_type_mismatch=0 the leak is reported:

```
Direct leak of 800 byte(s) in 1 object(s)
SUMMARY: AddressSanitizer: 800 byte(s) leaked in 1 allocation(s).
```

4.3 Repaired program

rate() and the destructor are virtual, override is used, the raw array is a std::vector, and accounts are held by pointer (unique_ptr), never copied.

```
#include <iostream>
#include <memory>
#include <vector>
class Account {
public:
    virtual double rate() const { return 3.5; }
    virtual ~Account() { std::cout << "~Account\n"; }
};
class SeniorAccount : public Account {
    std::vector<double> history = std::vector<double>(100);
public:
    double rate() const override { return 4.0; }
    ~SeniorAccount() override { std::cout << "~Senior\n"; }
};
int main() {
    std::vector<std::unique_ptr<Account>> v;
    v.push_back(std::make_unique<Account>());
    v.push_back(std::make_unique<SeniorAccount>());
    for (auto& a : v) std::cout << a->rate() << "\n";
}
```

Output:

```
3.5
4
~Account
~Senior
~Account
```

The senior account now earns 4 and every destructor runs (~Senior, then ~Account); ASan reports no errors.

4.4 Prevention rules

- A base class with virtual functions must have a virtual destructor.
- Mark every overriding function with override, and use pointers, references or smart pointers to base objects (never copy them by value) so slicing cannot occur.

Predict the Output

My predicted output: 3.5 4 3.5

```
#include <iostream>
struct Account {
    double rate() const { return 3.5; }
    virtual double vrate() const { return 3.5; }
    virtual ~Account() = default;
};
struct SeniorAccount : Account {
    double rate() const { return 4.0; }
    double vrate() const override { return 4.0; }
```

```
};
int main() {
    SeniorAccount s; Account* p = &s; Account copy = s;
    std::cout << p->rate() << ' ' << p->vrate() << ' ' << copy.vrate() << '\n';
}
```

Output:

```
3.5 4 3.5
```

p->rate() is non-virtual, so the call is bound at compile time using the pointer's static type (Account) and gives 3.5. p->vrate() is virtual, so it is bound at run time to SeniorAccount::vrate() and gives 4. copy is a separate Account object created by slicing, so copy.vrate() calls Account::vrate() and gives 3.5.

Alternative Concept and Complexity Analysis

std::vector<std::unique_ptr<Account>> (shown in 4.3) frees each object automatically when the vector goes out of scope.

Measure	Raw pointers + delete	unique_ptr in vector
Time to process n accounts	O(n)	O(n) (same)
Space	n pointers + n objects	same (unique_ptr is pointer-sized)
Defects still possible	all three	Leak by forgotten delete is gone. Wrong rate and wrong destructor (non-virtual) can still occur; slicing is impossible, as unique_ptr cannot be copied

TASK 5 – Pointers to Objects and the this Pointer

5.1 – 5.2 Four ways to reach an object, chaining and array-of-pointers search

(a) by name, with chaining via return *this; (b) pointer and ->; (c) reference; (d) new/delete. The array of Cart pointers is searched with pointer arithmetic.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
class Cart {
    double sum = 0;
public:
    Cart& add(double price) { sum += price; return *this; }
    double total() const { return sum; }
};
int main() {
    Cart a; a.add(10).add(20);                // (a) by name, chained
    Cart* p = &a; p->add(5);                  // (b) pointer
    Cart& r = a; r.add(5);                    // (c) reference
    Cart* d = new Cart; d->add(100); delete d; // (d) new / delete
    cout << a.total() << '\n';
    Cart* all[3] = { new Cart, new Cart, new Cart };
    all[0]->add(40); all[1]->add(90); all[2]->add(60);
    Cart** best = all;
    for (Cart** q = all + 1; q < all + 3; ++q) if ((*q)->total() > (*best)->total()) best = q;
    cout << "best index " << best - all << " total " << (*best)->total() << '\n';
    for (Cart* x : all) delete x;
    vector<Cart> v(1); Cart* first = &v[0]; v.reserve(1000);
    cout << (first == &v[0] ? "same" : "pointer invalidated") << '\n';
}
```

Output:

```
40
best index 1 total 90
pointer invalidated
```

5.3 Comparison

Way	Safety	Lifetime	Use when
Name	Safest	Automatic (scope)	Default choice for local objects
Pointer ->	Can be null or dangling	Not owned by pointer	Optional or reseatable access, arrays of objects
Reference	Cannot be null	Same as target	Parameters and aliases
new / delete	Leak or double delete risk	Manual, until delete	Object must outlive its scope (prefer smart pointers)

this is a pointer because it holds the address of the object on which the member function was called; *this is that object itself, so returning Cart& (return *this) lets the next call act on the same object.

Complete the Code (TODO 1 and 2)

```
Cart& add(const std::string& item, double price) {
    if (n < 10) { items[n] = item; prices[n++] = price; }
    return *this;
}
Cart& setDiscount(double discount) { this->discount = discount; return *this; }
```

```
#include <iomanip>
#include <iostream>
#include <string>
class Cart {
    std::string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;
public:
    Cart& add(const std::string& item, double price) {
        if (n < 10) { items[n] = item; prices[n++] = price; }
        return *this;
    }
    Cart& setDiscount(double discount) { this->discount = discount; return *this; }
    double total() const {
        double t = 0;
        for (int i = 0; i < n; ++i) t += prices[i];
        return t * (100 - discount) / 100;
    }
};
int main() {
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;
    p->add("Oil", 180);
    std::cout << std::fixed << std::setprecision(2) << p->total() << '\n';
}
```

Output:

```
324.00
```

If add() returned Cart (by value) instead of Cart&, each call would return a copy. Only the first add() acts on c; the later chained calls (add Dal, setDiscount) act on temporary copies and are lost, so the total would print 240.00 (Rice + Oil, no discount), not 324.00. Copying also costs time.

Alternative Concept and Complexity Analysis

std::vector<Cart> stores the objects themselves instead of pointers to them.

Measure	Array of Cart*	vector<Cart>
Time to find highest total	O(n)	O(n)

Space	n pointers + n separately allocated objects	n objects stored contiguously (copied in)
When it grows	Pointer array unaffected; objects stay in place	Reallocation moves all objects, so old pointers/references become invalid (shown by the 'pointer invalidated' line above)

TASK 6 – Design Challenge: Vehicle Rental Fleet

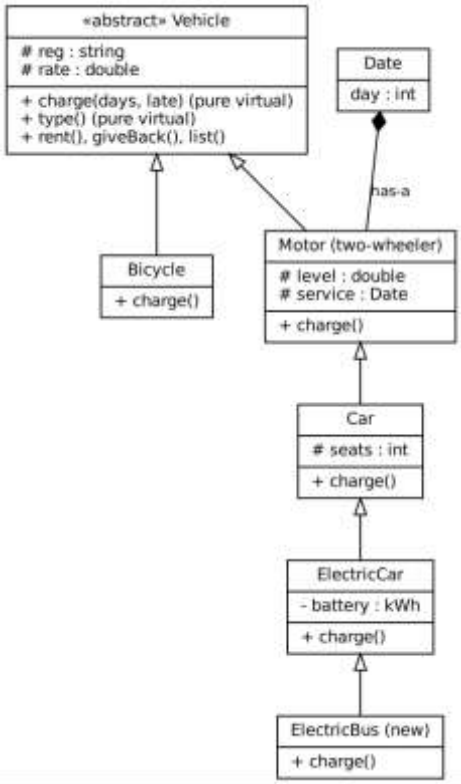
Stage 1 – Interpret the problem

Entities: Bicycle, two-wheeler, Car, ElectricCar (all are vehicles). Common data: registration number, daily rate. Motor vehicles add fuel/charge level and service due date. Car adds seating; ElectricCar adds battery capacity. Different behaviour: the rental charge.

Testable requirements:

- R1: A vehicle can be rented, returned and listed.
- R2: charge(days, late) gives a different result per type for the same days.
- R3: Returning late adds a late charge (late days × rate × factor).
- R4: An electric car is charged by battery kWh instead of fuel.
- R5: A new vehicle type can be added without editing any existing class.

Stage 2 – Hierarchy



Vehicle is abstract (pure virtual charge() and type()). Inheritance is multilevel: Vehicle → Motor → Car → ElectricCar → ElectricBus, and hierarchical at Vehicle (Bicycle and Motor). Date is a member class of Motor (has-a).

Stage 3 – Critical design decisions

- Vehicle holds reg, rate and the rented flag; rent(), giveBack() and list() are written once there.
- Motor holds level and service; Car adds seats; ElectricCar adds battery.
- charge() and type() are virtual (charge is pure virtual); the destructor is virtual.
- The fleet is stored as an array of Vehicle* so all vehicles are handled in one loop.

Stage 4 – Pseudocode

```
ALGORITHM RentVehicle(v)
BEGIN
    IF v.rented = TRUE THEN DISPLAY "not available" ; RETURN
    v.rented ← TRUE
END

ALGORITHM ReturnVehicle(v, days, lateDays)
BEGIN
    v.rented ← FALSE
    amount ← v.charge(days, lateDays)     // runtime binding picks the type's rule
    RETURN amount
END
```

Stage 5 – Implementation and demonstration

The program rents all five vehicles, returns each with 2 days and 1 late day, and ElectricBus is added at the end without any change to the other classes.

```
#include <iostream>
#include <string>
using namespace std;
struct Date { int day; };                                    // member class: service due date
class Vehicle {
protected:
    string reg; double rate; bool rented = false;
public:
    Vehicle(string r, double d) : reg(r), rate(d) {}
    virtual double charge(int days, int late) const = 0;     // pure virtual
    virtual string type() const = 0;
    virtual ~Vehicle() = default;
    void rent() { rented = true; }
    double giveBack(int days, int late) { rented = false; return charge(days, late); }
    void list() const { cout << reg << ' ' << type() << (rented ? " rented" : " free") << '\n'; }
};
class Bicycle : public Vehicle {
public:
    using Vehicle::Vehicle;
    double charge(int d, int l) const override { return rate * d + l * rate * 0.5; }
    string type() const override { return "Bicycle"; }
};
class Motor : public Vehicle {
protected:
    double level; Date service;
public:
    Motor(string r, double d, double lv, int sv) : Vehicle(r, d), level(lv), service{sv} {}
    double charge(int d, int l) const override { return rate * d + l * rate * 1.5 + 50; }   // +
fuel fee
    string type() const override { return "TwoWheeler"; }
};
class Car : public Motor {
protected:
    int seats;
public:
    Car(string r, double d, double lv, int sv, int s) : Motor(r, d, lv, sv), seats(s) {}
    double charge(int d, int l) const override { return rate * d + l * rate * 1.5 + 100; }
    string type() const override { return "Car"; }
};
class ElectricCar : public Car {
    double battery;
public:
    ElectricCar(string r, double d, double lv, int sv, int s, double b) : Car(r, d, lv, sv, s),
battery(b) {}
    double charge(int d, int l) const override { return rate * d + l * rate * 1.5 + battery * 8; }
// Rs 8 per kWh
    string type() const override { return "ElectricCar"; }
};
class ElectricBus : public ElectricCar {               // NEW type: no existing class changed
public:
    using ElectricCar::ElectricCar;
    double charge(int d, int l) const override { return ElectricCar::charge(d, l) * 2; }
```

```
    string type() const override { return "ElectricBus"; }
};
int main() {
    Vehicle* fleet[5] = { new Bicycle("B1", 100), new Motor("M1", 300, 80, 120), new Car("C1", 1500,
70, 150, 5),
                          new ElectricCar("E1", 2000, 90, 200, 5, 40), new ElectricBus("EB1", 5000,
95, 250, 40, 300) };
    for (Vehicle* v : fleet) { v->rent(); v->list(); }
    for (Vehicle* v : fleet) { cout << v->giveBack(2, 1) << '\n'; v->list(); }
    for (Vehicle* v : fleet) delete v;
}
```

Output:

```
B1 Bicycle rented
M1 TwoWheeler rented
C1 Car rented
E1 ElectricCar rented
EB1 ElectricBus rented
250
B1 Bicycle free
1100
M1 TwoWheeler free
5350
C1 Car free
7320
E1 ElectricCar free
39800
EB1 ElectricBus free
```

Alternative Concept and Complexity Analysis

```
#include <iostream>
#include <string>
struct Vehicle { std::string reg; double rate, level, battery; int type, seats, service; };   // all
fields in one class
double charge(const Vehicle& v, int d, int l) {
    switch (v.type) { case 0: return v.rate * d + l * v.rate * .5; case 1: return v.rate * d + l *
v.rate * 1.5 + 50;
                      case 2: return v.rate * d + l * v.rate * 1.5 + 100; default: return v.rate * d
+ l * v.rate * 1.5 + v.battery * 8; } }
int main() { Vehicle v{"E1", 2000, 90, 40, 3, 5, 200}; std::cout << charge(v, 2, 1) << ' ' <<
sizeof(Vehicle) << '\n'; }
```

Measure	Abstract base + virtual	Single class + switch
Time per charge()	O(1) (virtual call, vtable lookup)	O(1) (switch jump)
Space per object	only needed fields + one vtable pointer (8 bytes)	72 bytes for every object (measured), unused fields included
New vehicle type (electric bus)	1 new class, no existing class changed	edit the Vehicle struct, enum and the switch (existing code changed and retested)